

Act as an expert Senior Python & Machine Learning Instructor. I have attached a Jupyter Notebook (`.ipynb`)

I do not want high-level summaries or generic documentation. Instead, please provide an exhaustive, line-by-line, mechanical breakdown of every single operational line of code in this file.

For EVERY line of code (especially the mathematical steps, array manipulations, and plotting configurations), please address the following structural points:

1. **UNDERLYING MECHANICS & ARGUMENTS:** What do the specific function parameters or arguments mean at a low level? (For example, if a line reads ``axis=0``, ``full_matrices=False``, or ``[::-1]``, explain exactly what that specific configuration tells the computer to do step-by-step).
2. **STRUCTURAL & SHAPE TRANSFORMATION:** What are the input array shapes and data types before this line executes, and what is the exact output shape/data type immediately after it executes? (Explicitly use matrix dimensions like (N, D), (D, D), etc., to show how data is being stretched, collapsed, or transformed).
3. **BEHAVIORAL ALTERNATIVES:** What would happen to the code's behavior, execution flow, data shapes, and logic if those specific parameters were changed to other valid/invalid states? (Provide counter-examples of alternative parameter values and explain why they would either alter the outcome or break the script).
4. **ALGORITHMIC & MATH JUSTIFICATION:** Why is this exact configuration mathematically or structurally mandatory for the algorithm to function properly?

Please maintain this deep, parameter-focused engineering perspective throughout your entire line-by-line response.

Here is an exhaustive, parameter-focused engineering breakdown of every operational line of code in your Jupyter Notebook. As requested, this guide treats each statement with a low-level, mechanical perspective analyzing arguments, shapes, alternatives, and mathematical prerequisites.

CELL 1: GPU Verification

Line 1: `import tensorflow as tf`

1. **Underlying Mechanics & Arguments:** This line loads the TensorFlow framework into the Python runtime state, binding the module to the identifier `tf`. It instantiates core C++ runtimes (`libtensorflow_framework`) under the hood and starts background registration of system execution devices (CPUs and

GPUs via CUDA libraries).

2. **Structural & Shape Transformation:** No tensor transformations occur. Memory space is allocated within Sys-RAM to hold module references, class definitions (`tf.Tensor`, `tf.keras`), and internal APIs.
3. **Behavioral Alternatives:** `* import tensorflow as keras` would alias the whole library incorrectly, leading to namespace pollution or failure of subsequent down-stream syntax (`tf.config`).
4. **Algorithmic & Math Justification:** Mandatory to access high-performance numerical computation engines, automated graph execution graphs, and GPU-accelerated computing backends.

Line 2: `print("Num GPUs Available:",
len(tf.config.list_physical_devices('GPU')))`

1. **Underlying Mechanics & Arguments:** `* tf.config.list_physical_devices('GPU')` queries the visible environment hardware layer through the CUDA Management Library (NVML). The string argument `'GPU'` filters out host system devices (CPUs) and targets streaming multiprocessor hardware (e.g., your Colab T4).
 - `len(...)` computes the cardinality of the returned list.
2. **Structural & Shape Transformation:** Returns a standard Python list containing zero or more `PhysicalDevice` objects. In your environment, this wraps a single hardware pointer:
`[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]`. The `len()` wrapper casts this to a standard scalar Python integer (`int`), printed as 1.
3. **Behavioral Alternatives:** `* Passing 'CPU' as the argument returns [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU')]`.
 - Passing an invalid string like `'XPU'` yields an empty list `[]` instead of raising an error, because it evaluates string matching against known device types.
4. **Algorithmic & Math Justification:** Mechanically critical for ensuring that execution operations will run on accelerated hardware arrays instead of single-threaded or CPU-bound host threads, dramatically reducing processing times for 4D matrix operations.

CELL 2: Data Loading, Normalization, Reshaping, and One-Hot Encoding

Line 1-3: Import Statements

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
```

1. **Underlying Mechanics & Arguments:** These instructions extract localized API bindings out of the primary package workspace tree. `mnist` exposes the interface for remote raw-byte array storage files. `to_categorical` exposes a spatial matrix conversion function designed to map continuous scalar label classes onto orthogonal binary vector spaces.
2. **Structural & Shape Transformation:** Namespace operations only. No data array shapes or dimensions are mutated.
3. **Behavioral Alternatives:** Missing these specific imports causes runtime `NameError` faults when their down-stream wrappers are invoked.
4. **Algorithmic & Math Justification:** Prevents redundant code blocks by using pre-compiled C-extensions for disk I/O and categorical conversions.

Line 5: `(x_train, y_train), (x_test, y_test) = mnist.load_data()`

1. **Underlying Mechanics & Arguments:** Downloads a compressed serialized NumPy archive file (`mnist.npz`) from Google's storage URL via secure HTTP if not cached locally in `~/keras/datasets/`. Mechanically, it extracts the storage byte stream into memory, decompresses it, parses the binary structure, and unpacks it into two nested tuples representing training and evaluation subsets.
2. **Structural & Shape Transformation:** * `x_train`: Output shape `(60000, 28, 28)`, data type `uint8` (8-bit unsigned integers representing spatial luminosity from 0 to 255).
 - `y_train`: Output shape `(60000,)`, data type `uint8`.
 - `x_test`: Output shape `(10000, 28, 28)`, data type `uint8`.
 - `y_test`: Output shape `(10000,)`, data type `uint8`.
3. **Behavioral Alternatives:** Modifying the assignment structure (e.g., removing a tuple wrapper like `x, y = mnist.load_data()`) will raise a Python unpacking error: `ValueError: too many values to unpack (expected 2)`.

4. **Algorithmic & Math Justification:** Establishes the spatial data matrix foundations (X) and objective labels (Y) necessary to train a parametric machine learning classifier.

Line 7-8: Normalization

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

1. **Underlying Mechanics & Arguments:** * `.astype('float32')` creates an entirely new contiguous memory block converting every 8-bit unsigned element into a 32-bit IEEE 754 single-precision floating-point format.
 - `/ 255.0` executes an element-wise vector scaling division across the newly created continuous memory space.
2. **Structural & Shape Transformation:**
 - **Before:** Shape (60000, 28, 28) with `uint8` data values bounded between `[0, 255]`.
 - **After:** Shape (60000, 28, 28) with `float32` data values smoothly scaled between `[0.0, 1.0]`.
3. **Behavioral Alternatives:** * Omitting `.astype('float32')` and dividing directly by integer 255 in Python 3 forces implicit float upcasting but could risk memory boundary mismatches if standard NumPy types are passed to native low-level C++ wrappers.
 - Dividing by an integer 255 without a decimal point on integer arrays in purely custom NumPy implementations could trigger unexpected integer floor division (`//`), reducing all inputs to binary values 0 or 1.
4. **Algorithmic & Math Justification: Crucial for gradient stability.** Deep neural network weight initializers assume low-variance inputs bounded around small ranges (typically `[0, 1]` or `[-1, 1]`). Passing unnormalized value components up to 255.0 scales the intermediate activations exponentially, generating massive gradients that trigger numerical overflow, destabilizing model optimization (exploding gradient phenomena).

Line 11-12: Tensor Reshaping

```
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

- 1. Underlying Mechanics & Arguments:** Modifies the shape descriptor metadata within the NumPy array structure header without duplicating or re-arranging the underlying data values sequentially in the memory registers.
 - The `-1` argument tells the engine to automatically infer the necessary dimension size by evaluating the remaining data volume size against the explicitly defined dimensions.
 - `28, 28, 1` defines the spatial height, spatial width, and explicit channel quantity respectively.
- 2. Structural & Shape Transformation:**
 - **Before:** `x_train` shape was a 3D tensor `(60000, 28, 28)`.
 - **After:** `x_train` shape becomes a 4D tensor `(60000, 28, 28, 1)`. The total element count stays exactly the same ($60000 \times 28 \times 28 \times 1 = 47,040,000$). Data type remains `float32`.
- 3. Behavioral Alternatives:** * Changing the arguments to `(-1, 1, 28, 28)` alters the layout format to a channels-first structure (NCHW). This will break the model downstream because standard TensorFlow layers expect a channels-last structure (NHWC) by default.
 - Passing incorrect dimension dimensions (e.g., `(-1, 14, 14, 1)`) will raise a `ValueError: cannot reshape array of size 47040000 into shape (14,14,1)`.
- 4. Algorithmic & Math Justification: TensorFlow Conv2D layers require a 4D tensor input structured exactly as (Batch_Size, Height, Width, Channels).** Even though MNIST images are greyscale (single channel), the mathematical formulation of 2D cross-correlation algorithms requires an explicit channel dimension to compute structural matrix dot products over the network's spatial feature layers.

Line 15-16: Label Encoding

```
y_train = to_categorical(y_train, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)
```

- 1. Underlying Mechanics & Arguments:** Transforms a scalar classification value into a multi-dimensional row vector.
 - `num_classes=10` forces the output dimension array size to match the target label count (0 to 9). The routine reads each scalar label index i , instantiates a zero vector of length `num_classes`, and applies a binary `1.0` flag exclusively at target offset index i .
- 2. Structural & Shape Transformation:**

- **Before:** `y_train` was a 1D vector of shape `(60000,)` containing scalar class entries like `[5, 0, 4, ...]`.
 - **After:** `y_train` becomes a 2D matrix of shape `(60000, 10)` with a data type of `float32`. For instance, label integer 5 becomes `[0., 0., 0., 0., 0., 1., 0., 0., 0., 0.]`.
3. **Behavioral Alternatives:** * Setting `num_classes=12` forces the matrix output shape to `(60000, 12)`, which will result in two unused target indices at the end.
- Setting `num_classes=5` on data containing classes up to 9 raises an out-of-bounds error: `IndexError: index 5 is out of bounds for axis 1 with size 5`.
4. **Algorithmic & Math Justification: Required for cross-entropy optimization.** The final classification layer uses a 10-node softmax function that produces a probability distribution vector $\hat{y} \in \mathbb{R}^{10}$. To calculate categorical cross-entropy loss ($L = -\sum y_i \log \hat{y}_i$), the target ground-truth vector y must match those dimensions exactly so that the element-wise dot product can isolate and optimize the target class confidence score.

CELL 3: CNN Model Architecture Construction

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
```

- **Analysis:** Imports structural model containers and layer blueprints into the runtime execution context.

Line 4: `model = Sequential()`

1. **Underlying Mechanics & Arguments:** Instantiates an empty linear stack object. The Keras `Sequential` subclass creates an internal structural graph array where output tensors from a preceding block flow linearly as the direct inputs for the following layer.
2. **Structural & Shape Transformation:** Instantiates an empty network template object. No computational changes or matrix allocations occur yet.
3. **Behavioral Alternatives:** If you use alternative class models like `tf.keras.Model` (Functional API), you must explicitly define symbolic input nodes (`layers.Input`) and manually connect the layers as functional calls (e.g., `x = Conv2D()(inputs)`).

4. **Algorithmic & Math Justification:** Provides a clean wrapper that automatically handles data shape propagation across the network layer stack.

Line 6: First Convolutional Layer

```
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu',  
input_shape=(28, 28, 1)))
```

1. **Underlying Mechanics & Arguments:** * 32: The number of discrete spatial filters the layer will learn.
 - `kernel_size=(3, 3)`: The height and width dimensions of the sliding weight windows.
 - `activation='relu'`: Applies the element-wise Rectified Linear Unit transfer function: $f(x) = \max(0, x)$.
 - `input_shape=(28, 28, 1)`: Explicitly defines the shape of the incoming tensor, prompting the layer to allocate its internal kernel weight variables.
2. **Structural & Shape Transformation:**
 - **Input Shape:** (None, 28, 28, 1) where None represents a dynamic batch size variable.
 - **Output Shape:** (None, 26, 26, 32). The spatial grid shrinks from 28×28 to 26×26 because the valid sliding window positions for a 3×3 kernel across 28 pixels without padding is $28 - 3 + 1 = 26$. The channel depth expands from 1 to 32.
 - **Weights Generated:** A kernel tensor of shape (3, 3, 1, 32) and a bias vector of shape (32,). Total parameters: $(3 \times 3 \times 1 \times 32) + 32 = 320$.
3. **Behavioral Alternatives:** * Adding `padding='same'` keeps the spatial output at (None, 28, 28, 32) by automatically adding zero-value pixels along the boundaries.
 - Changing `activation='sigmoid'` replaces the ReLU function with $\sigma(x) = \frac{1}{1+e^{-x}}$, which can slow down training due to vanishing gradients in deeper networks.
4. **Algorithmic & Math Justification:** Local spatial feature maps are extracted via localized weight distributions. The spatial resolution parameters (3×3) capture fine details like edges and lines, while non-linearities (ReLU) ensure the network can approximate non-linear classification boundaries.

Line 7: First Pooling Layer

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

1. Underlying Mechanics & Arguments:

- `pool_size=(2, 2)`: Defines the spatial window dimensions used to downsample the feature maps. By default, the stride defaults to match the pool size dimensions, meaning the window moves 2 pixels at a time without overlapping. Within each window, it keeps only the maximum activation value.

2. Structural & Shape Transformation:

- **Input Shape:** `(None, 26, 26, 32)`.
- **Output Shape:** `(None, 13, 13, 32)`. The spatial height and width dimensions are cut in half ($\lfloor 26/2 \rfloor = 13$). The channel depth remains unchanged at 32.
- **Parameters:** 0. Pooling is a deterministic mathematical operation with no learnable parameters.

3. Behavioral Alternatives: * Changing to

`AveragePooling2D(pool_size=(2, 2))` calculates the mean instead of the maximum value, which can smooth out and dilute sharp, high-contrast feature activations.

- Setting `strides=(1, 1)` shifts the window by only one pixel at a time, which keeps the spatial resolution at `(None, 25, 25, 32)` and defeats the purpose of structural downsampling.

4. Algorithmic & Math Justification:

Reduces the spatial dimensionality of feature maps, which lowers the overall computational load and memory footprint. It also introduces spatial translation invariance, helping the network recognize key patterns even if their position shifts slightly.

Line 8: Second Convolutional Layer

```
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
```

1. Underlying Mechanics & Arguments:

- `64`: Specifies the number of unique feature filters to extract from the incoming representations.
- `kernel_size=(3, 3)`: The dimensions of each filter window.
- `activation='relu'`: Applies the element-wise rectifier function to the

calculated feature maps.

2. Structural & Shape Transformation:

- **Input Shape:** (None, 13, 13, 32).
- **Output Shape:** (None, 11, 11, 64). The spatial grid drops from 13×13 to 11×11 ($13 - 3 + 1 = 11$), while the channel depth expands to 64.
- **Weights Generated:** A kernel matrix tensor of shape (3, 3, 32, 64) and a bias vector of shape (64,). Total parameters: $(3 \times 3 \times 32 \times 64) + 64 = 18,496$.

3. **Behavioral Alternatives:** * If the input to this layer had a spatial shape smaller than the kernel size (e.g., (None, 2, 2, 32)), the execution would crash with a shape mismatch error because a 3×3 kernel cannot slide across a 2×2 matrix without padding.

4. **Algorithmic & Math Justification:** Allows the network to combine low-level features (like edges and curves from the first layer) into more complex, high-level structural representations (like loops and corners).

Line 9: Second Pooling Layer

```
model.add(MaxPooling2D(pool_size=(2, 2)))
```

1. **Underlying Mechanics & Arguments:** Downsamples the feature maps by applying a sliding 2×2 window with a stride of 2, isolating the maximum value in each window.

2. Structural & Shape Transformation:

- **Input Shape:** (None, 11, 11, 64).
- **Output Shape:** (None, 5, 5, 64). The spatial dimensions are halved with integer truncation ($\lfloor 11/2 \rfloor = 5$). Channel depth remains 64.
- **Parameters:** 0.

3. **Behavioral Alternatives:** Using a pool size of (3, 3) would reduce the dimensions even further to $\lfloor 11/3 \rfloor = 3$, accelerating spatial downsampling.

4. **Algorithmic & Math Justification:** Further refines spatial details, condensing the most salient features into a compact representations before passing them to the dense classification layers.

Line 11: `model.add(Flatten())`

1. **Underlying Mechanics & Arguments:** Unrolls a multi-dimensional tensor into a 1D vector for each sample in the batch, reshaping the data without altering its underlying order in memory.
2. **Structural & Shape Transformation:**
 - **Input Shape:** A 4D tensor of shape `(None, 5, 5, 64)`.
 - **Output Shape:** A 2D matrix of shape `(None, 1600)`. The spatial and channel dimensions are flattened into a single vector ($5 \times 5 \times 64 = 1600$).
 - **Parameters:** 0.
3. **Behavioral Alternatives:** * Replacing this with `GlobalAveragePooling2D()` calculates the average value across each channel's spatial grid (5×5), outputting a vector of shape `(None, 64)`. While this dramatically reduces the parameter count, it removes all spatial arrangement information, which can reduce accuracy if spatial layout is important.
4. **Algorithmic & Math Justification:** Serves as a bridge between structural feature-extracting layers (convolutional/pooling) and standard dense vector classifiers, which require flat vector inputs.

Line 13: Dense Hidden Layer

```
model.add(Dense(128, activation='relu'))
```

1. **Underlying Mechanics & Arguments:** * 128: Specifies the number of fully connected hidden neurons. Every input node connects to every neuron in this layer via an exclusive weight value.
 - `activation='relu'`: Applies the element-wise Rectified Linear Unit activation function.
2. **Structural & Shape Transformation:**
 - **Input Shape:** `(None, 1600)`.
 - **Output Shape:** `(None, 128)`.
 - **Weights Generated:** A weight matrix W of shape `(1600, 128)` and a bias vector b of shape `(128, )`. Total parameters: $(1600 \times 128) + 128 = 204,928$. The calculation follows standard matrix multiplication: $\text{Output} = \text{ReLU}(X \cdot W + b)$.
3. **Behavioral Alternatives:** Setting the size to a very large number like 4096

expands the parameter footprint significantly, which can lead to overfitting and slower training cycles if the dataset is small.

4. **Algorithmic & Math Justification:** Learns non-linear combinations of the high-level features extracted by the convolutional layers, serving as the primary reasoning layer of the classifier.

Line 14: Final Output Layer

```
model.add(Dense(10, activation='softmax'))
```

1. Underlying Mechanics & Arguments:

- `10`: One neuron for each potential class label (0 through 9).
- `activation='softmax'`: Normalizes the raw network outputs (logits) into a valid probability distribution. The softmax function handles this via:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

2. Structural & Shape Transformation:

- **Input Shape:** `(None, 128)`.
- **Output Shape:** `(None, 10)`.
- **Weights Generated:** A weight matrix of shape `(128, 10)` and a bias vector of shape `(10,)`. Total parameters: $(128 \times 10) + 10 = 1,290$.

3. **Behavioral Alternatives:** * Using a `linear` activation output instead of `softmax` changes the outputs to unbounded real numbers (logits), which prevents them from being interpreted directly as classification probabilities and causes errors when computing standard cross-entropy loss.
4. **Algorithmic & Math Justification:** Generates a structured, probability-based output vector where all elements are positive and sum to exactly `1.0`, enabling clear class predictions and stable loss calculations.

Line 17: `model.summary()`

1. **Underlying Mechanics & Arguments:** Loops through the registered layer sequence in the model object, reading shape configurations and parameter counts from the network's underlying graph representation before printing them out in a structured text table.
2. **Structural & Shape Transformation:** Diagnostic print only; does not modify

model parameters or array shapes.

CELL 4: Compilation and Training Loop execution

Line 3-7: Model Compilation

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

1. Underlying Mechanics & Arguments:

- `optimizer='adam'`: Instantiates the Adam (Adaptive Moment Estimation) optimization engine. Adam dynamically adjusts learning rates for individual parameters by tracking running averages of both the gradients (first moment) and the squared gradients (second moment).
- `loss='categorical_crossentropy'`: Configures the objective function used to measure model performance during training. It compares the predicted softmax distribution against the target one-hot encoded vector using the formula: $L = - \sum_i y_i \log(\hat{y}_i)$.
- `metrics=['accuracy']`: Tracks the percentage of correctly classified images at the end of each training epoch.

2. **Structural & Shape Transformation:** Compiles the underlying computational graph, mapping out the forward pass logic, the backward pass gradient derivations, and initializing the optimizer states. The model's primary shapes remain unchanged.

3. **Behavioral Alternatives:** * If your targets were raw integers instead of one-hot encoded vectors, using `categorical_crossentropy` would cause shape mismatches during loss computation. In that case, you would need to use `sparse_categorical_crossentropy`.

4. **Algorithmic & Math Justification:** Connects the structural network model with the optimization math and loss metrics required to calculate gradients and update model parameters during training.

Line 11-16: Training Loop Execution

```
history = model.fit(  
    x_train, y_train,  
    validation_data=(x_val, y_val),  
    epochs=10,  
    verbose=1
```

```
x_train,  
y_train,  
epochs=10,  
validation_data=(x_test, y_test)  
)
```

1. Underlying Mechanics & Arguments:

- `x_train, y_train`: The primary datasets used to train the network model.
- `epochs=10`: Specifies how many complete times the training process will cycle through the entire dataset. By default, Keras processes data in mini-batches of 32 samples. For 60,000 images, each epoch runs $\lceil 60000/32 \rceil = 1875$ gradient update steps.
- `validation_data=(x_test, y_test)`: Tells the engine to evaluate loss and accuracy on a separate test dataset at the end of each epoch to track generalization performance.

2. **Structural & Shape Transformation:** Runs forward and backward passes across mini-batches of size (32, 28, 28, 1). The process updates the model's 225,034 trainable weight parameters step-by-step, storing training metrics across all 10 epochs inside a `History` object container.

3. **Behavioral Alternatives:** Modifying the batch size (e.g., adding `batch_size=1024`) changes how many images are processed before each gradient update. Larger batch sizes speed up epoch execution times by utilizing parallel GPU processing more efficiently, but can require more memory and alter gradient dynamics.

4. **Algorithmic & Math Justification:** The core training loop where the network learns. By iteratively calculating losses and updating weights via backpropagation, the model minimizes classification errors and builds generalizable feature representations.

CELL 5: Evaluation, Metrics generation & Visualization Plots

Line 1-4: Evaluation Libraries Import

```
import numpy as np  
from sklearn.metrics import confusion_matrix,  
classification_report  
import matplotlib.pyplot as plt  
import sns as sns
```

-
- **Analysis:** Loads NumPy for numerical transformations, scikit-learn for evaluation metrics, and Matplotlib/Seaborn for data plotting.

Line 7: `test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)`

1. **Underlying Mechanics & Arguments:** Runs a forward inference pass over the evaluation dataset to measure performance. Setting `verbose=0` suppresses the standard progress text bar in the notebook output console.
2. **Structural & Shape Transformation:** Evaluates the test data arrays (`x_test` and `y_test`). It returns two standard Python float values (`test_loss` and `test_accuracy`) representing the final performance metrics.
3. **Behavioral Alternatives:** Setting `verbose=1` or `verbose=2` turns back on the visible processing progress logs without changing the underlying mathematical results.
4. **Algorithmic & Math Justification:** Provides an unbiased assessment of model performance on new, unseen data, helping detect if the network has overfitted to the training set.

Line 12: `y_pred_probs = model.predict(x_test)`

1. **Underlying Mechanics & Arguments:** Feeds the test dataset through the optimized network model graph to generate predictions.
2. **Structural & Shape Transformation:**
 - **Input Shape:** `(10000, 28, 28, 1)`.
 - **Output Shape:** `(10000, 10)` with a data type of `float32`. Each row contains a 10-element softmax probability vector representing the model's prediction confidence for each class.
3. **Behavioral Alternatives:** If you pass a single image array instead of a batch (e.g., shape `(28, 28, 1)`), the function will throw an error because it expects a 4D tensor that includes the batch dimension.
4. **Algorithmic & Math Justification:** Extracts raw prediction confidence vectors for every test image, which are required to calculate detailed performance metrics like confusion matrices.

Line 14-15: Label Vector Conversion

```
y_true_labels = np.argmax(y_test, axis=1)
y_pred_labels = np.argmax(y_pred_probs, axis=1)
```

1. **Underlying Mechanics & Arguments:** * `np.argmax(..., axis=1)` scans across the horizontal columns (axis index 1) of a 2D matrix to identify the position of the largest value in each row.
2. **Structural & Shape Transformation:**
 - **Before:** `y_test` and `y_pred_probs` were both 2D matrices of shape `(10000, 10)`.
 - **After:** `y_true_labels` and `y_pred_labels` are transformed into 1D vectors of shape `(10000,)` with an integer data type (`int64`), containing flat class index values from 0 to 9.
3. **Behavioral Alternatives:** Changing to `axis=0` scans vertically down the rows instead of across columns, which returns the index of the highest probability sample within each class column—breaking the evaluation logic entirely.
4. **Algorithmic & Math Justification:** **Converts probability and one-hot vectors back into categorical integer labels**, matching the input format required by standard scikit-learn evaluation metrics.

Line 18: `conf_matrix = confusion_matrix(y_true_labels, y_pred_labels)`

1. **Underlying Mechanics & Arguments:** Compares the actual target classes against the model's predicted labels, matching them element-by-element to count correct classifications and track where errors occurred.
2. **Structural & Shape Transformation:**
 - **Input Vectors:** Two 1D arrays of shape `(10000,)`.
 - **Output Matrix:** A 2D square matrix of shape `(10, 10)` with an integer data type (`int64`), where row positions represent the true classes and column positions represent predicted classes.
3. **Behavioral Alternatives:** Reversing the argument positions to `confusion_matrix(y_pred_labels, y_true_labels)` transposes the output matrix, swapping the axes for true labels and predicted labels.
4. **Algorithmic & Math Justification:** Provides a detailed, class-by-class look at model performance, showing exactly which digits are most frequently confused with one another.

Line 22: `print(classification_report(y_true_labels, y_pred_labels))`

1. **Underlying Mechanics & Arguments:** Analyzes the true and predicted labels to calculate key classification metrics like Precision ($\frac{TP}{TP+FP}$), Recall ($\frac{TP}{TP+FN}$), and F1-Score ($2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$) for each individual digit class.
2. **Structural & Shape Transformation:** Takes the 1D evaluation vectors and formats the resulting metrics into a clean, readable text summary table.
3. **Behavioral Alternatives:** Missing target classes in small datasets can cause divide-by-zero warnings, though scikit-learn handles these safely by defaulting the metric score for that missing class to 0.0.
4. **Algorithmic & Math Justification:** Offers a more nuanced look at performance than simple accuracy, ensuring the model performs well across all digit classes without harboring hidden biases.

Line 24: `plt.figure(figsize=(10, 8))`

1. **Underlying Mechanics & Arguments:** Instantiates a new figure drawing canvas object inside Matplotlib's backend manager workspace. `figsize=(10, 8)` sets the explicit dimensions of the target container layout window to 10 inches wide by 8 inches high.
2. **Structural & Shape Transformation:** Generates a blank canvas object container window layout; no array shape operations are performed.
3. **Behavioral Alternatives:** Omitting this statement defaults the canvas boundaries to a standard 6.4×4.8 inch layout, which can make dense confusion matrices look cramped or cut off text labels.
4. **Algorithmic & Math Justification:** Ensures the visualization has enough structural canvas area to display values clearly without overlapping labels.

Line 25-32: Seaborn Heatmap Visualization

```
sns.heatmap(
    conf_matrix,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=np.arange(10),
    yticklabels=np.arange(10))
```

)

1. Underlying Mechanics & Arguments:

- `conf_matrix`: The 10×10 integer evaluation matrix that supplies the data values for the plot.
- `annot=True`: Tells the engine to print the actual numeric count inside each individual grid square.
- `fmt='d'`: Forces the printed annotations to use decimal integer notation instead of scientific exponent notation (e.g., showing `980` instead of `9.8e+02`).
- `cmap='Blues'`: Uses a blue color gradient where darker shades represent higher numerical counts.
- `xticklabels/yticklabels=np.arange(10)`: Labels the horizontal and vertical grid axes with integer numbers from 0 to 9.

2. **Structural & Shape Transformation:** Maps the 2D confusion matrix array data onto a visual Seaborn plot, rendering it as a grid of colored pixels.

3. **Behavioral Alternatives:** Omitting `fmt='d'` when `annot=True` is enabled can cause text formatting issues, rendering large integer counts in scientific notation which makes the chart much harder to read.

4. **Algorithmic & Math Justification:** Converts raw numerical matrices into an intuitive visual format, making it easy to identify patterns of misclassification at a glance.

Line 33-36: Layout Labels & Render

```
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix for MNIST Classification')
plt.show()
```

1. **Underlying Mechanics & Arguments:** Sets the text labels for the x-axis, y-axis, and the main chart title before calling `plt.show()`. This flushes the rendered graphics buffer from internal memory and displays the final plot directly within your Jupyter notebook interface.

2. **Structural & Shape Transformation:** Finalizes the plot styling and clears the current figure from memory after rendering.

3. **Behavioral Alternatives:** Omitting `plt.show()` in some environments can cause the plot to display along with residual object memory address strings (like `<Figure size ...>`), or skip rendering entirely if run outside of an interactive

notebook environment.

4. **Algorithmic & Math Justification:** Adds necessary context to the chart, turning raw numbers into a clear, well-documented evaluation report.